EECE 7205: Fundamentals of Computer Engineering

Fall 2026

In-Class Work, Part 2: Binary Search Trees

Student Name: Natan Blumberg
Instructor: Dr. Fatema Nafa

In the tree drawings, [ ] marks the nodes compared on the path, in order from the root, ( ) marks a new node or a node that changed, and NULL marks where a failed search stops.

Problem 1: Building the Tree (Insert)

Case 3: Insert a larger key (insert 70)

(a) Rules I3 then I1. The tree is {50, 30}. Compare 70 with the root: $70 > 50$, so the key belongs in the right subtree of 50 (I3). The right child of 50 is NULL, so this is an empty spot and 70 is attached there as a leaf (I1). One comparison was needed. This is the mirror of Case 2: a smaller key goes left, a larger key goes right.

(b) Pseudocode.

```
1: cur <- root
2: if key > cur.key then
3:     if cur.right = NULL then
4:         cur.right <- newNode
5:         return
6:     cur <- cur.right
```

(c) Tree before and after.

```
Before: {50, 30}      After inserting 70

   50                   [50]
  /                     /  \
30                    30   (70)
```

70 is the right child of 50.

Case 4: Insert deep in the tree (insert 20, 40, 60, 80, then 35)

(a) Rules I2, I3, I2, then I1. First, 20 ($50 \rightarrow 30$, left of 30), 40 ($50 \rightarrow 30$, right of 30), 60 ($50 \rightarrow 70$, left of 70) and 80 ($50 \rightarrow 70$, right of 70) each take two comparisons and fill the second level. Then 35 is compared with 50, 30 and 40. $35 < 50$, go left (I2); $35 > 30$, go right (I3); $35 < 40$, go left (I2). The left child of 40 is NULL, so 35 is attached there as a leaf (I1). That is three comparisons, one per level.

35 goes left at one node and right at another because each step compares it with a different key: 35 is smaller than 50 and 40 but larger than 30. Each comparison only decides which side of that one node 35 belongs on, so the walk can turn back and forth. The result is that 35 lands between 30 and 40, which is its place in sorted order.

(b) Pseudocode (the loop that walks down).

```
 1: cur <- root
 2: loop
 3:     if key < cur.key then
 4:         if cur.left = NULL then
 5:             cur.left <- newNode
 6:             return
 7:         cur <- cur.left
 8:     else if key > cur.key then
 9:         if cur.right = NULL then
10:             cur.right <- newNode
11:             return
12:         cur <- cur.right
```

(c) Tree before and after inserting 35.

```
Before: 20, 40, 60, 80 inserted      After inserting 35

         50                                  [50]
       /    \                              /    \
   30         70                       [30]       70
  /  \       /  \                      /   \      /  \
20    40    60    80                 20     [40]  60   80
                                            /
                                         (35)
```

Before: 20, 40, 60 and 80 inserted. After: 35 is the left child of 40 (path $50 \to 30 \to 40$).

Case 5: Insert a key that is already in the tree (insert 40 again)

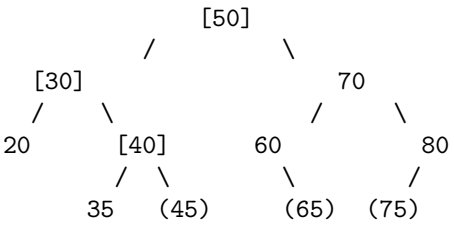
(a) Rules I2, I3, then I4. First the tree is finished: 45 ($50 \to 30 \to 40$, right of 40), 65 ($50 \to 70 \to 60$, right of 60) and 75 ($50 \to 70 \to 80$, left of 80). Then 40 is inserted again: $40 < 50$, go left (I2); $40 > 30$, go right (I3); $40 = 40$, so the key is already in the tree (I4). Nothing is inserted, the new node is thrown away and the tree does not change.

A BST must not store 40 twice because the BST property is strict: every key in a node's left subtree must be smaller than the node, and every key in its right subtree must be larger. A second 40 would have to go below the first 40, but it is neither smaller nor larger than 40, so either side breaks the rule. A search for 40 would also always stop at the first copy, so the second copy could never be found.

(b) Pseudocode.

```
1: if key < cur.key then ...
2: else if key > cur.key then ...
3: else
4:     discard newNode
5:     return
```

(c) Final tree of arr (all 11 keys).



( ): 45, 65 and 75, the last three inserts. [ ]: the path $50 \to 30 \to 40$ that stops at the existing 40 (I4).

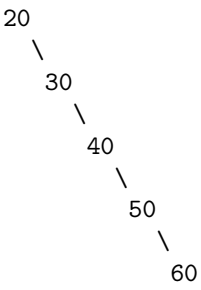
Case 6: Insert keys that arrive in sorted order (insert 20, 30, 40, 50, 60)

(a) Rule I3 every time. 20 becomes the root of the empty tree (I1). Every later key is larger than every key already in the tree, so at each node the comparison says go right (I3), until it reaches the NULL right child of the last node and is attached there (I1): 30 is the right child of 20, 40 of 30, 50 of 40 and 60 of 50. The tree is a chain going down to the right with height $4 = n - 1$. The Case 5 tree holds 11 keys with height 3, while this tree holds only 5 keys and is taller. No key ever goes left, so a search for 60 needs 5 comparisons, the same as walking a linked list.

(b) Pseudocode BUILD-FROM-ARRAY(data, n).

```
1: BUILD-FROM-ARRAY(data, n)
2: for i <- 0 to n - 1 do
3:     INSERT(data[i])
```

(c) Tree.



Sorted keys give a chain of height 4.

(g) Inserting all of arr

Step	Insert	Nodes compared (in order)	Rule(s)	Parent	Left / Right
1	50	none (tree is empty)	I1	none (root)	-
2	30	50	I2, I1	50	Left
3	70	50	I3, I1	50	Right
4	20	50, 30	I2, I2, I1	30	Left
5	40	50, 30	I2, I3, I1	30	Right
6	60	50, 70	I3, I2, I1	70	Left
7	80	50, 70	I3, I3, I1	70	Right
8	35	50, 30, 40	I2, I3, I2, I1	40	Left
9	45	50, 30, 40	I2, I3, I3, I1	40	Right
10	65	50, 70, 60	I3, I2, I3, I1	60	Right
11	75	50, 70, 80	I3, I3, I2, I1	80	Left

(h) Complete INSERT(key)

```
 1: INSERT(key)
 2: newNode <- new node with key
 3: if root = NULL then
 4:     root <- newNode
 5:     return
 6: cur <- root
 7: loop
 8:     if key < cur.key then
 9:         if cur.left = NULL then
10:             cur.left <- newNode
11:             return
12:         cur <- cur.left
13:     else if key > cur.key then
14:         if cur.right = NULL then
15:             cur.right <- newNode
16:             return
17:         cur <- cur.right
18:     else
19:         discard newNode
20:         return
```

The final tree has height 3. The deepest inserts are 35, 45, 65 and 75, which are placed at depth 3. Each needs 3 comparisons, one with a node on every level above it, so the deepest insert needs as many comparisons as the height of the tree.

Problem 2: Searching the Tree

All searches use the final tree of Problem 1.

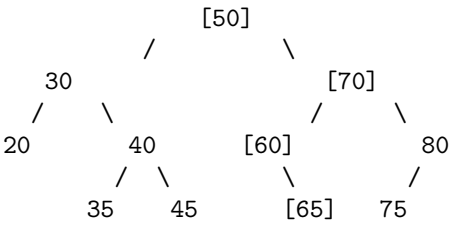
Case 3: The target is in the right subtree, path turns (search 65)

(a) Rules S4, S3, S4, S2. $65 > 50$, go right to 70 (S4). $65 < 70$, go left to 60 (S3). $65 > 60$, go right to 65 (S4). $65 = 65$, found (S2). That is 4 comparisons. The path turns right, then left, then right, because 65 is larger than 50, smaller than 70 and larger than 60, and at each node only one subtree can hold it. The left subtree of 50 and the subtree of 80 are never visited.

(b) Pseudocode.

```
1: cur <- root
2: while cur != NULL do
3:     if target = cur.key then return FOUND
4:     else if target < cur.key then cur <- cur.left
5:     else cur <- cur.right
```

(c) Search path.



Search path: $50 \to 70 \to 60 \to 65$. Found after 4 comparisons.

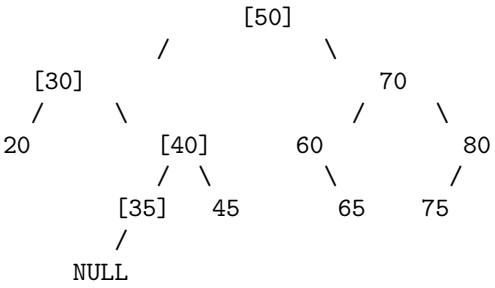
Case 4: The target is not in the tree (search 33)

(a) Rules S3, S4, S3, S3, S1. $33 < 50$, go left to 30 (S3). $33 > 30$, go right to 40 (S4). $33 < 40$, go left to 35 (S3). $33 < 35$, go left (S3), but 35 has no left child, so `cur` becomes NULL and the search stops: not found (S1). That is 4 comparisons. The search stops at the NULL left pointer of 35. If 33 were inserted, its parent would be 35 (as the left child), because insert follows the same path and attaches the new node at the same NULL spot.

(b) Pseudocode.

```
1: cur <- root
2: while cur != NULL do
3:     if target = cur.key then return FOUND
4:     else if target < cur.key then cur <- cur.left
5:     else cur <- cur.right
6: return NOT FOUND
```

(c) Search path.



Search path: $50 \to 30 \to 40 \to 35 \to$ NULL. Not found after 4 comparisons.

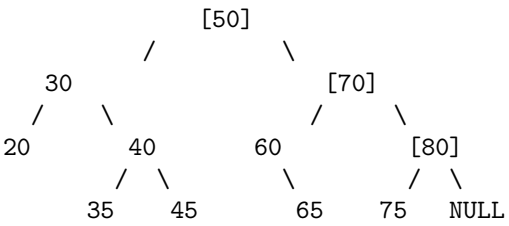
Case 5: The target is larger than every key (search 90)

(a) Rules S4, S4, S4, S1. $90 > 50$, go right to 70 (S4). $90 > 70$, go right to 80 (S4). $90 > 80$, go right (S4), but 80 has no right child, so `cur` becomes NULL: not found (S1). That is 3 comparisons. Because 90 is larger than every key, every comparison goes right. The search follows the rightmost path of the tree and stops just past the maximum, 80. If 90 were inserted, it would become the right child of 80.

(b) Pseudocode.

```
1: cur <- root
2: while cur != NULL do
3:     if target = cur.key then return FOUND
4:     else if target < cur.key then cur <- cur.left
5:     else cur <- cur.right
6: return NOT FOUND
```

(c) Search path.



Search path: $50 \to 70 \to 80 \to$ NULL. Not found after 3 comparisons.

Case 6: Find the minimum and the maximum key

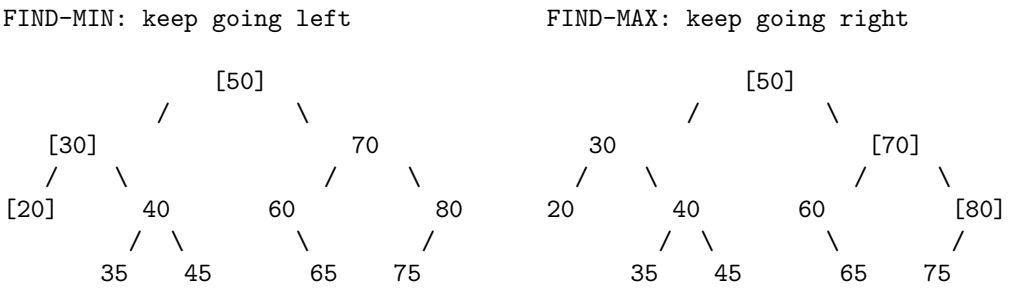
(a) Rule P2 (lecture notes): the minimum is the leftmost node and the maximum is the rightmost node. Every key smaller than a node is in its left subtree, so to find the smallest key, start at the root and keep following left pointers. Stop at the first node whose left child is NULL, because nothing smaller exists below it. Here $50 \to 30 \to 20$, and 20 has no left child, so the minimum is 20. For the maximum, keep following right pointers: $50 \to 70 \to 80$, and 80 has no right child, so the maximum is 80. 80 has a left child (75), but that does not matter because $75 < 80$. No key is compared with a target. Each step only checks whether a pointer is NULL, and each walk visits 3 nodes.

(b) Pseudocode FIND-MIN and FIND-MAX.

```
1: FIND-MIN()
2: if root = NULL then return EMPTY
3: cur <- root
4: while cur.left != NULL do
5:     cur <- cur.left
6: return cur.key
```

```
1: FIND-MAX()
2: if root = NULL then return EMPTY
3: cur <- root
4: while cur.right != NULL do
5:     cur <- cur.right
6: return cur.key
```

(c) Both paths.



Left: FIND-MIN visits $50 \to 30 \to 20$. Right: FIND-MAX visits $50 \to 70 \to 80$.

Summary table

Case	Target	Nodes compared (in order)	Rules	Result	Comparisons
1	50	50	S2	found	1
2	45	50, 30, 40, 45	S3, S4, S4, S2	found	4
3	65	50, 70, 60, 65	S4, S3, S4, S2	found	4
4	33	50, 30, 40, 35	S3, S4, S3, S3, S1	not found	4
5	90	50, 70, 80	S4, S4, S4, S1	not found	3
6	min	50, 30, 20 (visited)	P2	min = 20	0
	max	50, 70, 80 (visited)	P2	max = 80	0

(g) Largest number of comparisons

The largest number is 4. The tree has height 3, and its deepest nodes (35, 45, 65 and 75) are at depth 3, so a search that reaches one of them compares 4 nodes (45 in Case 2, 65 in Case 3). A failed search that falls off one of these deepest nodes also compares 4 nodes before reaching NULL (33 in Case 4). A search compares one node per level, so the worst case is $h + 1 = 4$ comparisons.

Problem 3: Deleting from the Tree

Every case starts again from the original final tree of Problem 1 (level order 50 30 70 20 40 60 80 35 45 65 75).

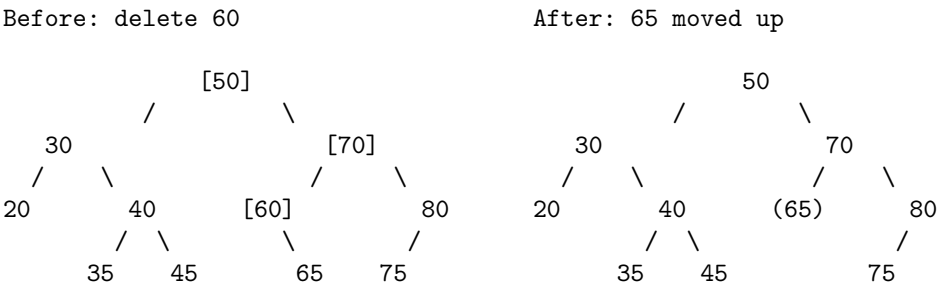
Case 3: Delete a node with only a right child (delete 60)

(a) Rule D2. Search: $60 > 50$, go right; $60 < 70$, go left; $60 = 60$, found. 60 has no left child and one right child, 65, so it is not a leaf (D1) but a node with one child (D2). 60 is replaced by its child: DELETE returns 65 to the call at 70, so 70's left pointer now points to 65 instead of 60, and 65 moves up one level. The BST property still holds, because 65 was already in the left subtree of 70 and in the right subtree of 50.

(b) Pseudocode.

```
1: else
2:     if node.left = NULL and node.right != NULL then
3:         return node.right
```

(c) Tree before and after.



60 has only a right child, 65. After the delete, 65 is the left child of 70.

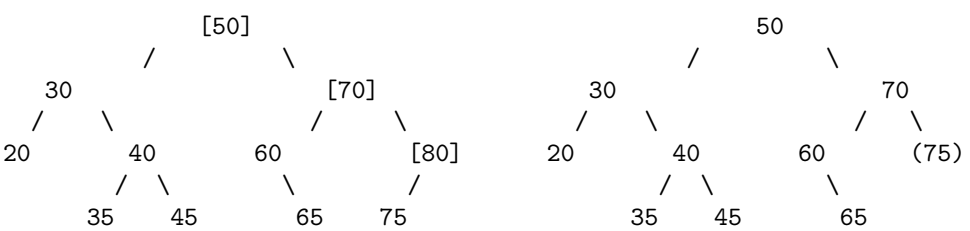
Case 4: Delete a node with only a left child (delete 80)

(a) Rule D2. Search: $80 > 50$, go right; $80 > 70$, go right; $80 = 80$, found. 80 has no right child and one left child, 75, so this is D2 again, mirrored from Case 3. 80 is replaced by its child: DELETE returns 75 to the call at 70, so 70's right pointer now points to 75. 75 is still larger than 70 and 50, so the BST property holds.

(b) Pseudocode.

```
1: else
2:     if node.right = NULL and node.left != NULL then
3:         return node.left
```

(c) Tree before and after.



80 has only a left child, 75. After the delete, 75 is the right child of 70.

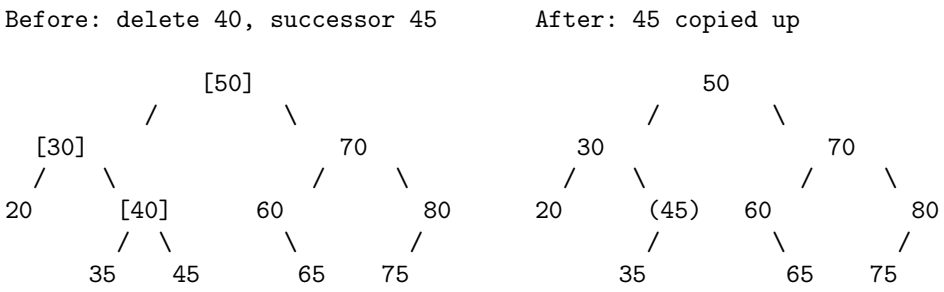
Case 5: Delete a node with two children, whose successor is its right child (delete 40)

(a) Rules D3 then D1. Search: $40 < 50$, go left; $40 > 30$, go right; $40 = 40$, found. 40 has two children, 35 and 45, so this is D3. The inorder successor is the smallest key in the right subtree: go right once to 45, then left as far as possible. 45 has no left child, so the "go left" step takes zero steps and the successor is 45. It is 40's own right child because 40's right subtree is only $\{45\}$: when the right child has no left subtree, nothing in the right subtree is smaller than it. 45 is copied into the node, so the node now holds 45. Then 45 is deleted from the node's right subtree. The old 45 is a leaf, so it is removed with D1 and the node's right pointer becomes NULL. The result has 45 as the right child of 30, with 35 as its left child. Inorder 20 30 35 45 50 60 65 70 75 80 is still sorted.

(b) Pseudocode.

```
1: else
2:     s <- node.right
3:     while s.left != NULL do
4:         s <- s.left
5:     node.key <- s.key
6:     node.right <- DELETE(node.right, s.key)
7:     return node
```

(c) Tree before and after.



The node that held 40 now holds 45, and the old 45 leaf is gone.

Case 6: Delete the root, whose successor has a child of its own (delete 50)

(a) Rules D3 then D2. Search: $50 = 50$ at the root, found after one comparison. 50 has two children, 30 and 70, so this is D3. Successor: go right once to 70, then left as far as possible. $70 \to 60$, and 60 has no left child, so stop: the successor is 60. 60 is copied into the root. Then 60 is deleted from the right subtree: DELETE(70, 60) goes left from 70 and finds the old 60. The old 60 has no left child but has a right child, 65, so it is deleted with D2, and 70's left pointer now points to 65. The successor can never have a left child (otherwise we would have kept going left), so deleting it is always D1 or D2. Here it is D2 because 60 has a right child. The new root is 60, and inorder 20 30 35 40 45 60 65 70 75 80 is still sorted.

(b) Pseudocode.

```
1: else
2:     s <- node.right
3:     while s.left != NULL do
4:         s <- s.left
5:     node.key <- s.key
6:     node.right <- DELETE(node.right, s.key)
7:     return node
```

(c) Tree before and after.

```
Before: delete 50, successor 60       After: 60 copied up, 65 moved up

             [50]                                  (60)
           /      \                              /      \
   30                  70                30                70
  /    \              /    \            /    \            /    \
20       40       60          80      20       40      (65)      80
        /  \       \         /                /  \              /
      35    45      65     75                35    45          75
```

The successor walk goes right to 70, then left to 60. After the delete, 60 is the root and 65 is the left child of 70.

Summary table

Case	Delete	Children of the node	Successor	Rules applied	Level order after
1	33	not found	-	S1	50 30 70 20 40 60 80 35 45 65 75
2	20	none	-	D1	50 30 70 40 60 80 35 45 65 75
3	60	one (right: 65)	-	D2	50 30 70 20 40 65 80 35 45 75
4	80	one (left: 75)	-	D2	50 30 70 20 40 60 75 35 45 65
5	40	two (35, 45)	45	D3, D1	50 30 70 20 45 60 80 35 65 75
6	50	two (30, 70)	60	D3, D2	60 30 70 20 40 65 80 35 45 75

(g) Complete DELETE(node, key)

```
 1: DELETE(node, key)
 2: if node = NULL then
 3:     return NULL
 4: if key < node.key then
 5:     node.left <- DELETE(node.left, key)
 6: else if key > node.key then
 7:     node.right <- DELETE(node.right, key)
 8: else
 9:     if node.left = NULL and node.right = NULL then
10:         return NULL
11:     else if node.left = NULL then
12:         return node.right
13:     else if node.right = NULL then
14:         return node.left
15:     else
16:         s <- node.right
17:         while s.left != NULL do
18:             s <- s.left
19:         node.key <- s.key
20:         node.right <- DELETE(node.right, s.key)
21: return node
```

(h) Why copy the inorder successor?

In a general binary tree the nodes have no order, only a shape, so any node can fill the hole. The deepest, rightmost node is used because removing it keeps the tree complete. In a BST the position of a key is decided by its value. The key that replaces a node with two children must be larger

than every key in the node's left subtree and smaller than every key in its right subtree. Only the keys right next to it in sorted order, the inorder successor (or the predecessor), satisfy this. The deepest node usually does not. In this tree the deepest, rightmost node is 75. Moving 75 into the root to delete 50 gives inorder 20 30 35 40 45 75 60 65 70 80, which is not sorted, because 60, 65 and 70 are now in the right subtree of 75 but are smaller than 75. Copying the successor 60 keeps every key on the correct side. The successor also has no left child, so removing it is always the easy case D1 or D2, and the whole delete stays on one path down the tree, $O(h)$.